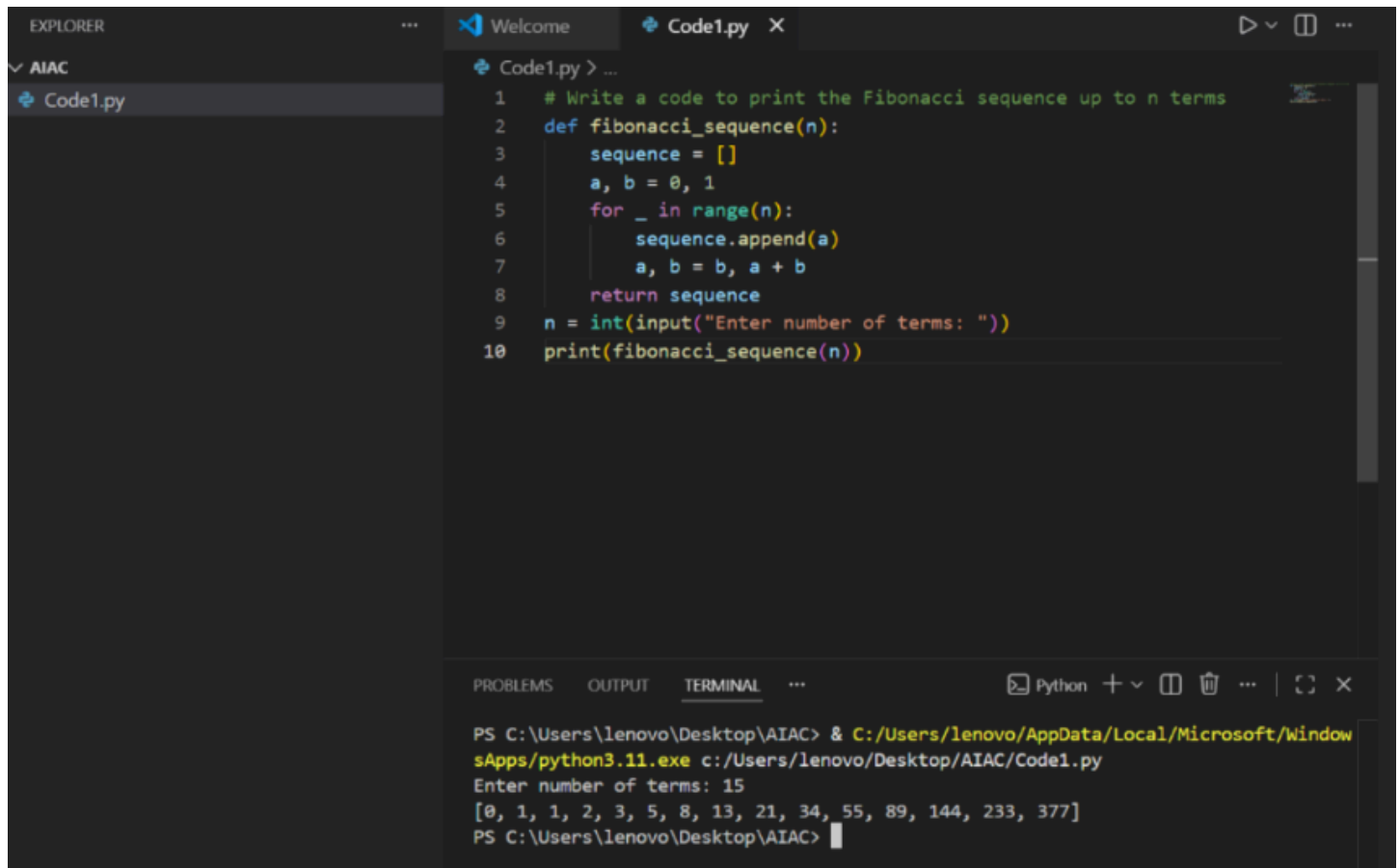


AI Assisted Coding Assignment - 1

Chilukala Sai Architha(2303A52220) - Batch 37

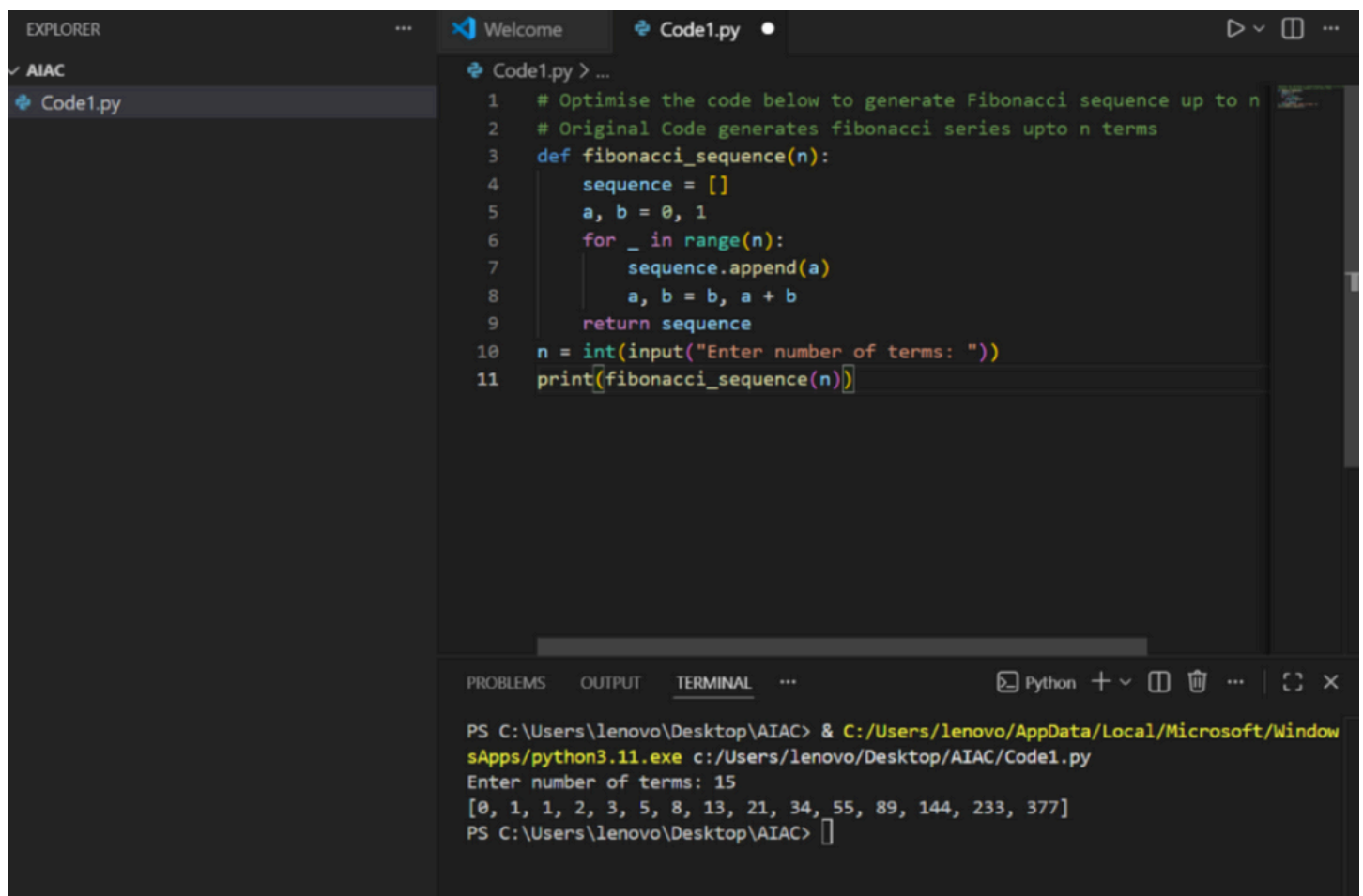


This screenshot shows the first implementation of a Fibonacci sequence generator in a Python file named `Code1.py`. The code defines a function `fibonacci_sequence(n)` that takes an integer `n` as input and returns a list of the first `n` terms of the Fibonacci sequence. The sequence starts with 0 and 1, and each subsequent term is the sum of the two preceding terms. The code prompts the user to enter the number of terms and prints the resulting sequence.

```
1 # Write a code to print the Fibonacci sequence up to n terms
2 def fibonacci_sequence(n):
3     sequence = []
4     a, b = 0, 1
5     for _ in range(n):
6         sequence.append(a)
7         a, b = b, a + b
8     return sequence
9 n = int(input("Enter number of terms: "))
10 print(fibonacci_sequence(n))
```

The terminal output shows the execution of the code:

```
PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py
Enter number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS C:\Users\lenovo\Desktop\AIAC>
```

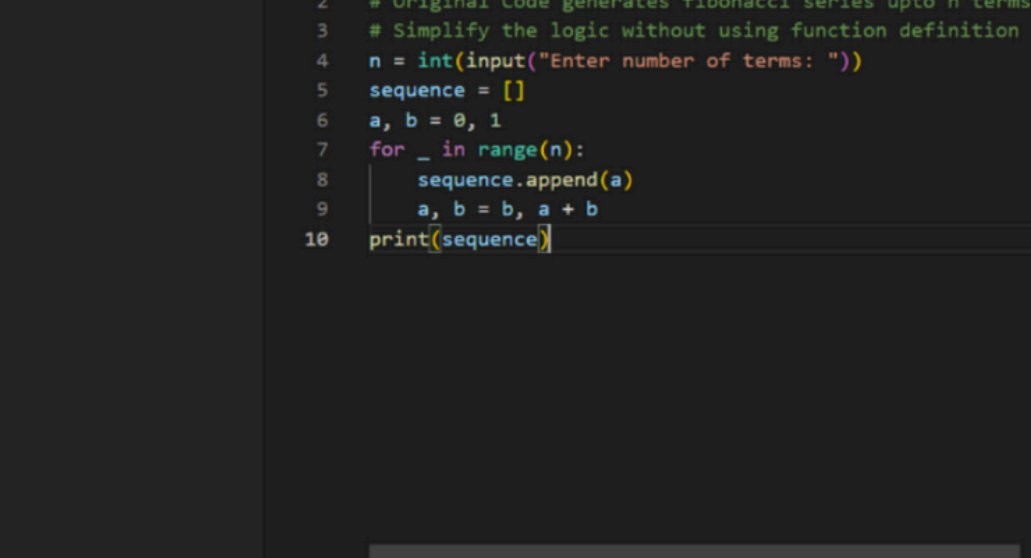


This screenshot shows the second implementation of a Fibonacci sequence generator in a Python file named `Code1.py`. The code is identical to the first implementation, but it includes a comment indicating that it is an optimized version of the original code. The code defines a function `fibonacci_sequence(n)` that takes an integer `n` as input and returns a list of the first `n` terms of the Fibonacci sequence. The sequence starts with 0 and 1, and each subsequent term is the sum of the two preceding terms. The code prompts the user to enter the number of terms and prints the resulting sequence.

```
1 # Optimise the code below to generate Fibonacci sequence up to n
2 # Original Code generates fibonacci series upto n terms
3 def fibonacci_sequence(n):
4     sequence = []
5     a, b = 0, 1
6     for _ in range(n):
7         sequence.append(a)
8         a, b = b, a + b
9     return sequence
10 n = int(input("Enter number of terms: "))
11 print(fibonacci_sequence(n))
```

The terminal output shows the execution of the code:

```
PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py
Enter number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS C:\Users\lenovo\Desktop\AIAC>
```



```
1  # Optimise the code below to generate Fibonacci sequence up to n
2  # Original Code generates fibonacci series upto n terms
3  # Simplify the logic without using function definition
4  n = int(input("Enter number of terms: "))
5  sequence = []
6  a, b = 0, 1
7  for _ in range(n):
8      sequence.append(a)
9      a, b = b, a + b
10 print(sequence)
```

PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py

Enter number of terms: 15

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]

PS C:\Users\lenovo\Desktop\AIAC>

```
AIAC
Code1.py

Code1.py > ...
1 # Optimise the code below to generate Fibonacci sequence up to n terms
2 # Original Code generates fibonacci series upto n terms
3 # Simplify the logic without using function definition
4 # Fibonacci code using user defined function
5 def fibonacci_sequence(n):
6     sequence = []
7     a, b = 0, 1
8     for _ in range(n):
9         sequence.append(a)
10        a, b = b, a + b
11    return sequence
12 n = int(input("Enter number of terms: "))
13 print(fibonacci_sequence(n))
14
```

PROBLEMS OUTPUT TERMINAL ... Python + - [] [X] [Y]

```
PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py
Enter number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS C:\Users\lenovo\Desktop\AIAC>
```

EXPLORER

AIAC

Code1.py

```
Code1.py > ...
1 # Optimise the code below to generate Fibonacci sequence up to n
2 # Original Code generates fibonacci series upto n terms
3 # Simplify the logic without using function definition
4 # Fibonacci code using user defined function
5 # An iterative Fibonacci implementation
6
7 n = int(input("Enter number of terms: "))
8 sequence = []
9 a, b = 0, 1
10 for _ in range(n):
11     sequence.append(a)
12     a, b = b, a + b
13 print(sequence)
```

PROBLEMS OUTPUT TERMINAL ... Python + -

```
PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py
Enter number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS C:\Users\lenovo\Desktop\AIAC>
```

EXPLORER

AIAC

Code1.py

```
Code1.py > ...
1 # Optimise the code below to generate Fibonacci sequence up to n
2 # Original Code generates fibonacci series upto n terms
3 # Simplify the logic without using function definition
4 # Fibonacci code using user defined function
5 # An iterative Fibonacci implementation
6 # A recursive Fibonacci implementation
7 def fibonacci_recursive(n):
8     if n <= 1:
9         return n
10    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
11
12 n = int(input("Enter number of terms: "))
13 for i in range(n):
14     print(fibonacci_recursive(i), end=" ")
15
```

PROBLEMS OUTPUT TERMINAL ... Python + -

```
PS C:\Users\lenovo\Desktop\AIAC> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/AIAC/Code1.py
Enter number of terms: 15
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
PS C:\Users\lenovo\Desktop\AIAC>
```

Time and Space Complexity Comparison

Approach	Time Complexity	Space Complexity
Iterative Fibonacci	$O(n)$	$O(1)$
Recursive Fibonacci	$O(2^n)$	$O(n)$

Performance for Large n

Aspect	Iterative	Recursive
Execution speed	Very fast	Very slow
Memory usage	Minimal	High (call stack)
Scalability	Excellent	Poor
Risk of crash	None	Stack overflow

Conclusion

- Iterative Fibonacci works efficiently even for large values like $n = 10000$.
- Recursive Fibonacci becomes extremely slow and may crash for values above $n = 40$.